Title: Assignment 4

Query Design for ECommerce Database-Commerce Database

Zhushan He

Selvi Lesmana Putri

Alhagie Kijera

Isaac Ebu-Danso

Date: 13.02.2026

1. Introduction

This assignment presents 20 queries designed for an ecommerce relational database. The schema models customers, sellers, products, orders, payments, reviews, and geolocations. The assignment is divided into two parts. Part 1 contains 10 basic queries expressed in natural language, relational algebra, and SQL. Part 2 contains 10 complex queries expressed in natural language and SQL, including joins, aggregations, subqueries, grouping, and set operations.-commerce relational database. The schema models customers, sellers, products, orders, payments, reviews, and geolocations. The assignment is divided into two parts. Part 1 contains 10 basic queries expressed in natural language, relational algebra, and SQL. Part 2 contains 10 complex queries expressed in natural language and SQL, including joins, aggregations, subqueries, grouping, and set operations.

2. Schema Overview

The database includes the following relations:

- **Customer(customer_id, customer_name, customer_zip_code)**
- **Seller(seller_id, seller_name, seller_zip_code)**
- **Geolocation(geo_zip_code, geo_city, geo_state)**

- **Order(order_id, order_date, customer_id, order_status, order_delivery_date, order_estimation_delivery_date)**
- **Order_item(order_item_id, order_id, product_id, seller_id, quantity, total_price_per_product)**
- **Order_review(order_review_id, order_id, order_review_score, order_review_comment)**
- **Order_payment(payment_id, order_id, payment_amount, payment_type)**
- **Product(product_id, product_category_id, product_name, product_price_per_unit, stock_quantity)**
- **Product_category(product_category_id, product_category_name)**

3. Part 1 Basic Queries (10)

Each query includes natural language, relational algebra, and SQL.

Query 1 — List all customers

Natural Language: Retrieve all customers stored in the database.

Relational Algebra: $\pi_{customer_id,customer_name,customer_zip_code}(Customer)$

SQL:

```
SELECT customer_id,
       customer_name,
       customer_zip_code
FROM Customer
ORDER BY customer_name;
```

Query 2 — Show all products with their category names

Natural Language: Display all products with their category names.

Relational Algebra: $Product \bowtie_{product_category_id} Product_category$

SQL:

```
SELECT p.product_id,
       p.product_name,
       c.product_category_name
FROM Product p
JOIN Product_category c ON p.product_category_id =
c.product_category_id;
```

Query 3 — Retrieve all orders placed by customer with ID = 10

Natural Language: Find all orders made by customer 10. **including their city**.

Relational Algebra: π order_id, customer_name, geo_city (σ customer_id=10 (Order ⋈ Customer ⋈ Geolocation))

SQL:

```
SELECT c.customer_name,
       g.geo_city,
       o.order_id,
       o.order_date,
       o.order_status,
       o.order_deliver_date,
       o.order_approval_date,
       o.order_estimation_delivery_date
FROM "Order" o
JOIN Customer c ON o.customer_id = c.customer_id
JOIN Geolocation g ON c.customer_zip_code =
g.geo_zip_code
WHERE o.customer_id = 10;
```

Query 4 — List all sellers and their city/state

Natural Language: Show sellers with their city and state.

Relational Algebra: Seller⋈seller_zip_code=geo_zip_codeGeolocation

SQL:

```
SELECT s.seller_id,
       s.seller_name,
       g.geo_city,
       g.geo_state
FROM Seller s
JOIN Geolocation g ON s.seller_zip_code =
g.geo_zip_code;
```

Query 5 — Show all order items for order ID = 5

Natural Language: Retrieve all items of order ID 5, showing product name and seller name.

Relational Algebra: π order_item_id, product_name, seller_name (σ order_id=5 (Order_item ⋈ Product ⋈ Seller))

SQL:

```
SELECT oi.order_id,
       oi.product_id,
       p.product_name,
       oi.seller_id,
       s.seller_name,
       oi.quantity,
       oi.total_price_per_product
FROM Order_item oi
JOIN Product p ON oi.product_id = p.product_id
```

```
JOIN Seller s ON oi.seller_id = s.seller_id
WHERE oi.order_id = 5;
```

Query 6 — List all reviews with their order dates

Natural Language: Show reviews with the date of the related order.

Relational Algebra: Order_review⋈Order

SQL:

```
SELECT r.order_review_id,
       r.order_review_score,
       o.order_date
FROM Order_review r
JOIN "Order" o ON r.order_id = o.order_id;
```

Query 7 — Show all payments and their order IDs

Natural Language: Retrieve all payments and display the customer name who paid.

Relational Algebra:

π payment_id, payment_amount, customer_name (Order_payment ⋈ Order ⋈ Customer)

SQL:

```
SELECT op.payment_id,
       op.payment_amount,
       c.customer_name
FROM Order_payment op
JOIN "Order" o ON op.order_id = o.order_id
JOIN Customer c ON o.customer_id = c.customer_id;
```

Query 8 — List all products sold by seller with ID = 3

Natural Language: Find products sold by seller 3.

Relational Algebra: $\sigma_{seller_id=3}(Order_item) \bowtie Product$

SQL:

```
SELECT DISTINCT p.product_id,
                p.product_name
FROM Order_item oi
JOIN Product p ON oi.product_id = p.product_id
WHERE oi.seller_id = 3;
```

Query 9 — Show all customers and the city they live in

Natural Language: Display customers with their city.

Relational Algebra: $Customer \bowtie Geolocation$

SQL:

```
SELECT c.customer_id,
       c.customer_name,
       g.geo_city
FROM Customer c
JOIN Geolocation g ON c.customer_zip_code =
g.geo_zip_code;
```

Query 10 — List all orders with customer names

Natural Language: Retrieve orders with customer names.

Relational Algebra: $Order \bowtie Customer$

SQL:

```
SELECT o.order_id,
       o.order_date,
       c.customer_name
FROM "Order" o
JOIN Customer c ON o.customer_id = c.customer_id;
```

4. Part 2 Complex Queries (10)

Natural language + SQL only.

Query 11 — Products by Category with Stock Info

List all products with their category names and stock status.

SQL:

```
SELECT p.product_id,
       p.product_name,
       pc.product_category_name,
       p.product_price_per_unit,
       p.stock_quantity,
       CASE
           WHEN p.stock_quantity = 0 THEN 'Out of
Stock'
           WHEN p.stock_quantity < 10 THEN 'Low
Stock'
           ELSE 'In Stock'
       END AS stock_status
FROM Product p
JOIN Product_Category pc ON p.product_category_id =
pc.product_category_id
```

```
ORDER BY pc.product_category_name,
         p.product_name;
```

Query 12 — Top 5 bestselling products by quantity-selling products by

Identify the top 5 bestselling products based on total quantity sold, including their category names.

SQL:

```
SELECT p.product_name,
       pc.product_category_name,
       SUM(oi.quantity) AS total_sold
FROM Order_item oi
JOIN Product p ON oi.product_id = p.product_id
JOIN Product_category pc ON p.product_category_id =
pc.product_category_id
GROUP BY p.product_name,
         pc.product_category_name
ORDER BY total_sold DESC
LIMIT 5;
```

Query 13 — Customers who never placed an order

Find customers who are registered in the database but have never placed an order.

SQL:

```
SELECT customer_id,
       customer_name
FROM Customer c
WHERE NOT EXISTS
     (SELECT 1
      FROM "Order" o
      WHERE o.customer_id = c.customer_id);
```

Query 14 — Average review score per product

Calculate the average review score for each product.

SQL:

```
SELECT p.product_name,
       AVG(r.order_review_score) AS avg_score
FROM Order_review r
JOIN "Order" o ON r.order_id = o.order_id
JOIN Order_item oi ON o.order_id = oi.order_id
JOIN Product p ON oi.product_id = p.product_id
GROUP BY p.product_name;
```

Query 15 — Total revenue per seller

Calculate the total revenue generated by each seller.

SQL:

```
SELECT s.seller_id,
       s.seller_name,
       SUM(oi.total_price_per_product) AS revenue
FROM Order_item oi
JOIN Seller s ON oi.seller_id = s.seller_id
GROUP BY s.seller_id,
         s.seller_name;
```

Query 16 — Orders delivered later than estimated

List all orders that were delivered after their estimated delivery date, including the location of the customer.

SQL:

```
SELECT o.order_id,
       c.customer_name,
```

```
       g.geo_city
FROM "Order" o
JOIN Customer c ON o.customer_id = c.customer_id
JOIN Geolocation g ON c.customer_zip_code =
g.geo_zip_code
WHERE o.order_delivery_date >
o.order_estimation_delivery_date;
```

Query 17 — Customer Order Summary

Get order count and total spent per customer.

SQL:

```
SELECT c.customer_id,
       c.customer_name,
       COUNT(o.order_id) AS total_orders,
       COUNT(DISTINCT o.order_id) AS unique_orders,
       MIN(o.order_date) AS first_order_date,
       MAX(o.order_date) AS last_order_date
FROM Customer c
LEFT JOIN "Order" o ON c.customer_id = o.customer_id
GROUP BY c.customer_id,
         c.customer_name
ORDER BY total_orders DESC;
```

Query 18 — Sellers and customers in the same city (UNION)

List all unique names of sellers and customers along with their cities.

SQL:

```
SELECT seller_name AS name,
       g.geo_city
FROM Seller s
```

```
JOIN Geolocation g ON s.seller_zip_code =
g.geo_zip_code

UNION

SELECT customer_name AS name,
       g.geo_city
FROM Customer c
JOIN Geolocation g ON c.customer_zip_code =
g.geo_zip_code;
```

Query 19 — Order with payment information

Retrieve orders per customer which successfully delivered with their total payment

SQL:

```
SELECT c.customer_name,
       o.order_id,
       o.order_date,
       o.order_status,
       SUM(op.payment_amount) AS total_paid
FROM "Order" o
JOIN Customer c ON o.customer_id = c.customer_id
LEFT JOIN Order_Payment op ON o.order_id =
op.order_id
WHERE o.order_status IN ('delivered',
                         'shipped')
GROUP BY c.customer_name,
         o.order_id,
         o.order_date,
         o.order_status;
```

Query 20 — Most expensive product in each category

Find the most expensive product in each product category.

SQL:

```
SELECT p.product_category_id,
       p.product_name,
       p.product_price_per_unit
FROM Product p
WHERE p.product_price_per_unit =
    (SELECT MAX(product_price_per_unit)
     FROM Product
     WHERE product_category_id =
p.product_category_id);
```

Query 21 — Products Never Ordered

Find products that have never been ordered.

SQL:

```
SELECT p.product_id,
       p.product_name,
       p.product_price_per_unit,
       p.stock_quantity,
       pc.product_category_name
FROM Product p
JOIN Product_Category pc ON p.product_category_id =
pc.product_category_id
LEFT JOIN Order_Item oi ON p.product_id =
oi.product_id
WHERE oi.product_id IS NULL
ORDER BY p.product_name;
```

Query 22 — Seller Performance Summary

Calculate total sales and order count per seller.

SQL:

```
SELECT s.seller_id,
       s.seller_name,
       COUNT(DISTINCT oi.order_id) AS total_orders,
       SUM(oi.quantity) AS total_items_sold,
       ROUND(SUM(oi.total_price_per_product), 2) AS
total_revenue
FROM Seller s
LEFT JOIN Order_Item oi ON s.seller_id = oi.seller_id
GROUP BY s.seller_id,
         s.seller_name;
```

Query 23 — Top 10 Customers by Revenue with Rankings

Rank customers by total spending with revenue breakdown.

SQL:

```
SELECT c.customer_id,
       c.customer_name,
       COUNT(DISTINCT o.order_id) AS total_orders,
       SUM(oi.quantity) AS total_items_purchased,
       ROUND(SUM(oi.total_price_per_product), 2) AS
total_spent,
       ROUND(AVG(oi.total_price_per_product), 2) AS
avg_order_value,
       RANK() OVER (
                     ORDER BY
SUM(oi.total_price_per_product) DESC) AS
revenue_rank,
```

```
       ROUND(SUM(oi.total_price_per_product) /
SUM(SUM(oi.total_price_per_product)) OVER () * 100,
2) AS percentage_of_total_revenue
FROM Customer c
JOIN "Order" o ON c.customer_id = o.customer_id
JOIN Order_Item oi ON o.order_id = oi.order_id
GROUP BY c.customer_id,
         c.customer_name
ORDER BY total_spent DESC
LIMIT 10;
```

Query 24 — Product Category Performance with Inventory Alerts

Product category analysis with stock warnings.

SQL:

```
WITH CategoryStats AS
  (SELECT pc.product_category_id,
          pc.product_category_name,
          COUNT(DISTINCT p.product_id) AS
total_products,
          SUM(p.stock_quantity) AS total_stock,
          ROUND(AVG(p.product_price_per_unit), 2) AS
avg_price,
          COUNT(DISTINCT oi.order_id) AS
total_orders,
          COALESCE(SUM(oi.quantity), 0) AS
total_units_sold,
```

```
COALESCE(ROUND(SUM(oi.total_price_per_product), 2),
0) AS total_revenue
   FROM Product_Category pc
   JOIN Product p ON pc.product_category_id =
p.product_category_id
   LEFT JOIN Order_Item oi ON p.product_id =
oi.product_id
   GROUP BY pc.product_category_id,
            pc.product_category_name)
SELECT product_category_id,
       product_category_name,
       total_products,
       total_stock,
       avg_price,
       total_orders,
       total_units_sold,
       total_revenue,
       revenue_per_product,
       avg_units_per_product,
       inventory_status,
       revenue_rank
FROM CategoryStats
```

```
ORDER BY total_revenue DESC;
```

Query 25 — Order Fulfillment Performance Analysis

Analyze delivery performance with delays and on-time metrics.

SQL:

```
SELECT o.order_id,
       c.customer_name,
       s.seller_name,
       o.order_date,
       o.order_approval_date,
       o.order_delivery_date,
       o.order_estimation_delivery_date,
       EXTRACT(DAYS
               FROM (o.order_approval_date -
o.order_date)) AS approval_days,
       EXTRACT(DAYS
               FROM (o.order_delivery_date -
o.order_approval_date)) AS delivery_days,
       EXTRACT(DAYS
               FROM (o.order_delivery_date -
o.order_estimation_delivery_date)) AS delay_days,
       CASE
           WHEN o.order_delivery_date IS NULL THEN
'Not Delivered'
           WHEN o.order_delivery_date <=
o.order_estimation_delivery_date THEN 'On Time'
           WHEN o.order_delivery_date <=
o.order_estimation_delivery_date + INTERVAL '2 days'
THEN 'Slight Delay'
```

```
            ELSE 'Late Delivery'
        END AS delivery_status,
        ROUND(SUM(oi.total_price_per_product), 2) AS
order_value
FROM "Order" o
JOIN Customer c ON o.customer_id = c.customer_id
JOIN Order_Item oi ON o.order_id = oi.order_id
JOIN Seller s ON oi.seller_id = s.seller_id
WHERE o.order_date >= CURRENT_DATE - INTERVAL '90
days'
GROUP BY o.order_id,
         c.customer_name,
         s.seller_name,
         o.order_date,
         o.order_approval_date,
         o.order_delivery_date,
         o.order_estimation_delivery_date
ORDER BY o.order_date DESC;
```

Query 26 — Seller Geographic Distribution and Performance

Analyze seller performance by geographic location.

SQL:

```
SELECT g.geo_state,
       g.geo_city,
       COUNT(DISTINCT s.seller_id) AS total_sellers,
       COUNT(DISTINCT oi.order_id) AS total_orders,
       ROUND(AVG(oi.total_price_per_product), 2) AS
avg_order_value,
```

```
       ROUND(SUM(oi.total_price_per_product), 2) AS
total_revenue,
       ROUND(SUM(oi.total_price_per_product) /
SUM(SUM(oi.total_price_per_product)) OVER () * 100,
2) AS percentage_of_total_revenue,
       DENSE_RANK() OVER (
                           ORDER BY
SUM(oi.total_price_per_product) DESC) AS
city_revenue_rank
FROM Geolocation g
JOIN Seller s ON g.geo_zip_code = s.seller_zip_code
LEFT JOIN Order_Item oi ON s.seller_id = oi.seller_id
GROUP BY g.geo_state,
         g.geo_city
HAVING COUNT(DISTINCT oi.order_id) > 0
ORDER BY total_revenue DESC;
```

Query 27 — Inventory Turnover and Restock Recommendations

Calculate inventory turnover ratio and recommend restock.

SQL:

```
WITH ProductSales AS
  (SELECT p.product_id,
          p.product_name,
          pc.product_category_name,
          p.stock_quantity,
          p.product_price_per_unit,
          COALESCE(SUM(oi.quantity), 0) AS
total_sold_90days
   FROM Product p
```

```
   JOIN Product_Category pc ON p.product_category_id
= pc.product_category_id
   LEFT JOIN Order_Item oi ON p.product_id =
oi.product_id
   LEFT JOIN "Order" o ON oi.order_id = o.order_id
   AND o.order_date >= CURRENT_DATE - INTERVAL '90
days'
   GROUP BY p.product_id,
            p.product_name,
            pc.product_category_name,
            p.stock_quantity,
            p.product_price_per_unit)
SELECT product_id,
       product_name,
       product_category_name,
       stock_quantity,
       total_sold_90days,
       ROUND(total_sold_90days / 3.0, 2) AS
avg_monthly_sales,
       ROUND(stock_quantity::DECIMAL /
NULLIF(total_sold_90days / 3.0, 0), 2) AS
months_of_inventory,
       ROUND((total_sold_90days / 3.0) * 1.5 -
stock_quantity, -- 1.5 months safety stock
0) AS recommended_restock_quantity,
       CASE
           WHEN stock_quantity = 0 THEN 'OUT OF STOCK
- URGENT'
           WHEN stock_quantity < (total_sold_90days /
3.0) THEN 'LOW - Restock Soon'
```

```
            WHEN stock_quantity > (total_sold_90days /
3.0 * 3) THEN 'OVERSTOCKED'
            ELSE 'ADEQUATE'
        END AS inventory_status,
        ROUND(stock_quantity * product_price_per_unit,
2) AS inventory_value
FROM ProductSales
WHERE total_sold_90days > 0
ORDER BY CASE
              WHEN stock_quantity = 0 THEN 1
              WHEN stock_quantity < (total_sold_90days
/ 3.0) THEN 2
              ELSE 3
          END,
          total_sold_90days DESC;
```

Query 28 — Inventory Alert System (UNION)

Combining different types of inventory issues.

SQL:

```
-- Critical Stock Alerts
SELECT 'INVENTORY_CRITICAL' AS alert_type,
        1 AS priority,
        p.product_id AS entity_id,
        p.product_name AS entity_name,
        pc.product_category_name AS category,
        p.stock_quantity AS current_value,
        0 AS threshold_value,
        'Out of stock - Immediate action required' AS
alert_message,
        CURRENT_TIMESTAMP AS alert_date
```

```
FROM Product p
JOIN Product_Category pc ON p.product_category_id =
pc.product_category_id
WHERE p.stock_quantity = 0

UNION ALL -- Low Stock Warnings

SELECT 'INVENTORY_LOW' AS alert_type,
       2 AS priority,
       p.product_id AS entity_id,
       p.product_name AS entity_name,
       pc.product_category_name AS category,
       p.stock_quantity AS current_value,
       10 AS threshold_value,
       CONCAT('Low stock - Only ', p.stock_quantity,
' units remaining') AS alert_message,
       CURRENT_TIMESTAMP AS alert_date
FROM Product p
JOIN Product_Category pc ON p.product_category_id =
pc.product_category_id
WHERE p.stock_quantity > 0
  AND p.stock_quantity < 10

UNION ALL -- Delayed Orders

SELECT 'ORDER_DELAYED' AS alert_type,
       1 AS priority,
       o.order_id AS entity_id,
       c.customer_name AS entity_name,
       o.order_status AS category,
       EXTRACT(DAYS
```

```
                FROM (CURRENT_DATE -
o.order_date))::INTEGER AS current_value,
       7 AS threshold_value,
       CONCAT('Order delayed - ', EXTRACT(DAYS
                                          FROM
(CURRENT_DATE - o.order_date)), ' days since order,
still ', o.order_status) AS alert_message,
       o.order_date AS alert_date
FROM "Order" o
JOIN Customer c ON o.customer_id = c.customer_id
WHERE o.order_status IN ('processing',
                         'shipped')
  AND o.order_date < CURRENT_DATE - INTERVAL '7 days'

UNION ALL -- Overdue Deliveries

SELECT 'DELIVERY_OVERDUE' AS alert_type,
       1 AS priority,
       o.order_id AS entity_id,
       c.customer_name AS entity_name,
       'Delivery' AS category,
       EXTRACT(DAYS
                FROM (CURRENT_DATE -
o.order_estimation_delivery_date))::INTEGER AS
current_value,
       0 AS threshold_value,
       CONCAT('Delivery overdue by ', EXTRACT(DAYS
                                              FROM
(CURRENT_DATE - o.order_estimation_delivery_date)), '
days') AS alert_message,
```

```
       o.order_estimation_delivery_date AS alert_date
FROM "Order" o
JOIN Customer c ON o.customer_id = c.customer_id
WHERE o.order_delivery_date IS NULL
  AND o.order_estimation_delivery_date < CURRENT_DATE
  AND o.order_status NOT IN ('cancelled',
                             'delivered')

UNION ALL -- Products with Low Review Scores

SELECT 'QUALITY_ISSUE' AS alert_type,
       2 AS priority,
       p.product_id AS entity_id,
       p.product_name AS entity_name,
       pc.product_category_name AS category,
       COALESCE(ROUND(AVG(orv.order_review_score),
2), 0) AS current_value,
       3 AS threshold_value,
       CONCAT('Low average rating: ',
COALESCE(ROUND(AVG(orv.order_review_score), 2), 0), '
stars from ', COUNT(orv.order_id), ' reviews') AS
alert_message,
       MAX(orv.order_review_date) AS alert_date
FROM Product p
JOIN Product_Category pc ON p.product_category_id =
pc.product_category_id
JOIN Order_Item oi ON p.product_id = oi.product_id
JOIN Order_Review orv ON oi.order_id = orv.order_id
WHERE orv.order_review_date >= CURRENT_DATE -
INTERVAL '90 days'
```

```
GROUP BY p.product_id,
         p.product_name,
         pc.product_category_name
HAVING AVG(orv.order_review_score) < 3.0
AND COUNT(orv.order_id) >= 5
ORDER BY priority ASC,
         alert_date DESC;
```

Query 29 — Find Customers Who Left Reviews (INTERSECT)

List down customers who placed orders and wrote reviews.

SQL:

```
-- Customers who placed orders
SELECT DISTINCT customer_id, customer_name
FROM Customer c
JOIN "Order" o ON c.customer_id = o.customer_id

INTERSECT

-- Customers who wrote reviews
SELECT DISTINCT c.customer_id, c.customer_name
FROM Customer c
JOIN "Order" o ON c.customer_id = o.customer_id
JOIN Order_Review orv ON o.order_id = orv.order_id;
```

5. Conclusion

This assignment demonstrates how to translate information needs into relational algebra and SQL queries. The queries cover basic retrieval, joins, grouping, aggregation, subqueries, and set operations. The exercise deepened understanding of relational database querying and the

structure of our ecommerce schema. The SQL script contain following specifications which thoroughly analyze the interdependencies between all tables in e-commerce schema database:-commerce schema.

- 10 queries with basic complexity level in natural language and relational algebra
- 19 queries with medium to advanced complexity level (e.g., involving aggregations) in natural language
- Specification of overall 28 queries:
  - 27 queries apply joins syntax
  - 15 queries involves more than two relations
  - 3 queries apply set operations (UNION, UNION ALL, INTERSECT)
  - 12 queries with different aggregation functions (SUM, COUNT, AVG, MIN, MAX)
  - 2 queries use subqueries
  - 12 queries use GROUP BY